

CSC3002F – Networks Assignment 1 – 2023

Socket programming project

Department of Computer Science

University of Cape Town

Due Date: 6 March 2023



GROUP 68

Fabio O’Ryan-Paulo (ORYFAB001)

Yachyaa Toefy (TFYYAC001)

Luke Clayton (CLYLUK001)

Design and functionality

Fabio O’Ryan-Paulo client implementation

Firstly, I created a dictionary called commands which associates a command to a certain number. The client has four possible commands, namely, “exit”, “upload”, “download” and “list”. The dictionary is defined as follows:

```
commands = {"exit": "0", "upload": "1", "download": "2", "list": "3"}
```

The server accepts numbers as commands, but I believe that commands in the form of words are more meaningful and memorable. Next, the user is prompted to enter the host name and port number of the server that they’re trying to connect to. A TCP client socket is then created which is used to form the client-server connection. If a connection is successfully established, a welcome message and a list of possible commands are printed to the screen. The user is then prompted to enter one of the four commands. The “exit” command terminates the client-server connection, “upload” allows the client to upload a public or password protected file, “download” allows the client to download a file stored on the server, and “list” prints a list of all the files stored on the server with their privacy information and byte size. To implement clarity and accessibility to my client, I have made use of the “>” character. If a user is visually impaired the “>” character helps indicate that user input is required. It is also a common command line convention. I’ve wrapped my client code in a while loop so that the client can send multiple commands per session. Additionally, if the user enters an invalid command, the loop goes to the next iteration until a valid command is entered.

Yachyaa Toefy client implementation

The client connects to the server via TCP connections using the sockets library. The client first requests the IP address and port number of the server that you want to connect to. Once a successful connect has been established, the client requests the user to pick an action to be performed on the server.

I have decided to get the user request using numbers, where each number represents a specific request to the server.

1 = upload, 2 = download, 3 = “list” etc.

The user request is then sent to the server. Depending on the request, the user is prompted to perform an action:

- Upload – user is requested to enter a file path and an optional file key.
- Download – user is prompt to enter the filename and if the file is private, the user is prompted to enter a file key.
- List – no user action required.
- Exception text are displayed depending on the input of the user.
- Upload - if file does not exist on local pc: "File not found.\n"
 - if file already exists on server: "File already exists on server.\n"
- Download – if file is not in database "File not found on database\n"
 - if incorrect file key inputted "Unable to retrieve file. Incorrect key provided.\n"
- list - no user action required.

Luke Clayton client implementation

I created a client which connects to the server through a TCP connection with the aid of the socket library. It makes use of a dictionary to link commands with letters as follows:

This is done to simplify the commands and it makes it more efficient for the user to insert commands.

q - Exit program

u - Upload file to the server

d - Download file from the server

l - List available files on server

```
Greetings! Welcome to the File Transfer Protocol!
Commands:
q - Exit program
u - Upload file to the server
d - Download file from the server
l - List available files on server

Please enter a command:
-->
```

If the action that the user request is not available, the client sends appropriate error messages back to the client. For example, this can happen when a file that the user requests does not exist.

```
Please enter a command:
--> u
Please enter file path of the file you would like to upload:
--> fileExist.txt
Error: The file that you are trying to upload cannot be found.
```

List of Features

Upload

The upload feature allows users to upload files from their local device to the server. We have decided to make use of a database to store the different element of the file and the file key.

On the client's side, the uploading process starts with the client asking a user to enter a file path and the client program checks if that file path exists. If the file path exists, the client program will ask the user to enter in a key. If the user enters no key, the file will be stored as public and if the user enters a key, the file will be stored as protected. The file, and the file key will be sent to the server.

On the server side, the server will receive the file and extract the different elements of the file (file name, file content, size). The server will also receive the file key and will perform a hashing and salting algorithm to encrypt the file key. All the file elements and the file key will be stored in a SQL database. If the file has a key the file will be stored as private on the database and if the file does not have a file key, it will be stored as public.

Download

The download feature will allow the user to download files from the server to the client's local device.

On the client's side, the client program will prompt the user to enter the file name of the file that the user wants to download.

The server will receive the file name and it will check if a file with a matching file name exists in the database. If the file is not existing, it will return the Boolean "False" to the user.

If the file is found, the server will check whether the file is public or private. If the file is public, it will send the file and its content to the user. If the file is private, the server will send a request to the client for a file key. The user on the client program will be prompt to enter a file key and the

inputted file key will be sent back to the server. The server will then apply the same hashing and salting encryption algorithm on the users inputted file key. If the users encrypted file key matches the stored file key, the file name and content will be sent to the client. If the file key does not match, the file will not be sent.

List

When listing the elements in the database, the server will first fetch all the elements in the database. The data will be stored in a 2d array, where each outer element represents the rows (files) in the database and each inner element represents the attribute of each row/file (file name, file content, file key, file size, privacy).

All the data will be stored in a single list variable.

Result = { [privacy1, fileName1, fileSize1,], [privacy2, fileName2, fileSize2] }

Before adding the data to the list variable, the server checks whether the file has a file key. If the file has a file key, the file data will be added with its privacy listed as private. If the file does not have a file key, the file data will be added with its privacy as public.

The outputted list that the client receives will be formatted as follows:

```
[private]      cats.jpeg      [48829 Bytes]
[private]      file1.txt [0 Bytes]
[private]      file2.txt [73 Bytes]
[private]      file3.txt [0 Bytes]
```

Protocol Specification

Message Format

Sending protocol: Whenever a file/message is sent between the client and the server, a 4-byte header is applied to file sent which indicated how big the file is in bytes. We have implemented this protocol using the “struct” library in python. The entire message is structured as followed:

(xxxx + message) - xxxx indicating the file size in 4 bytes.

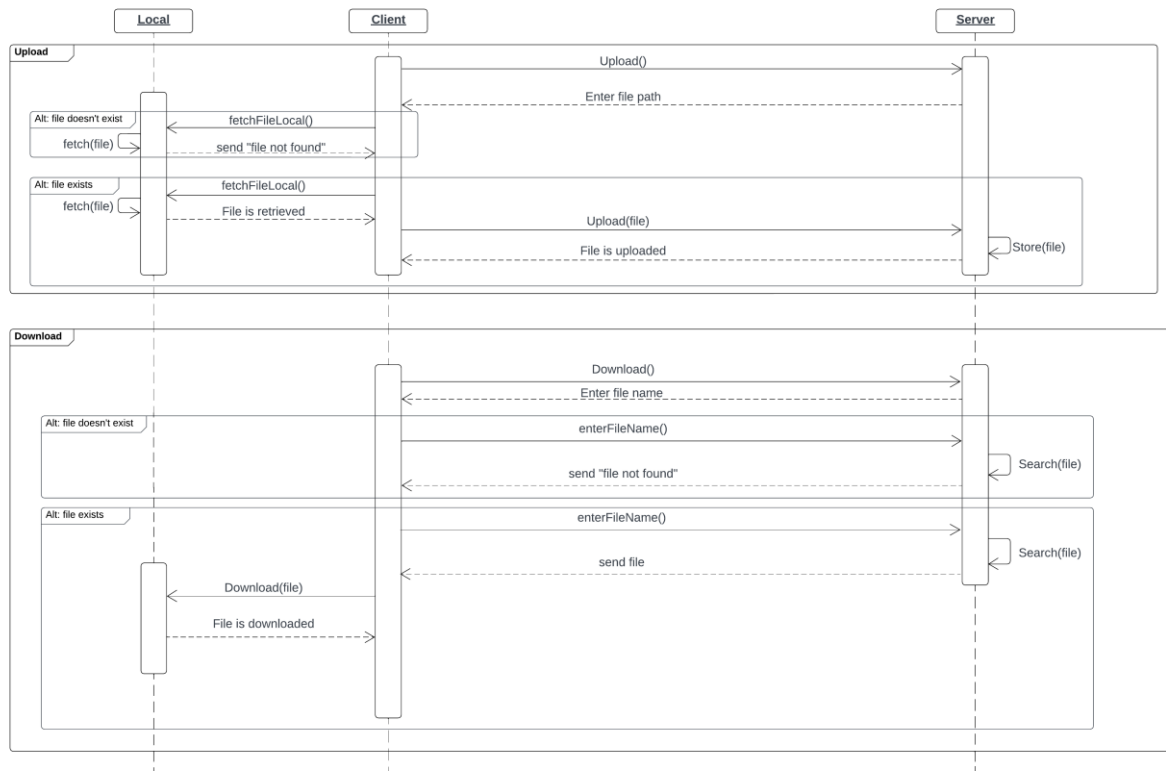
Receiving protocol: The full message (xxxx + message) is received. Once the message is received, the 4-byte file size is extracted from the message and stored as “n”. The rest of the message is received and the “recv” function will only read “n” number of bytes after the header. The message extracted is then stored in its own variable and can then be processed.

Structure

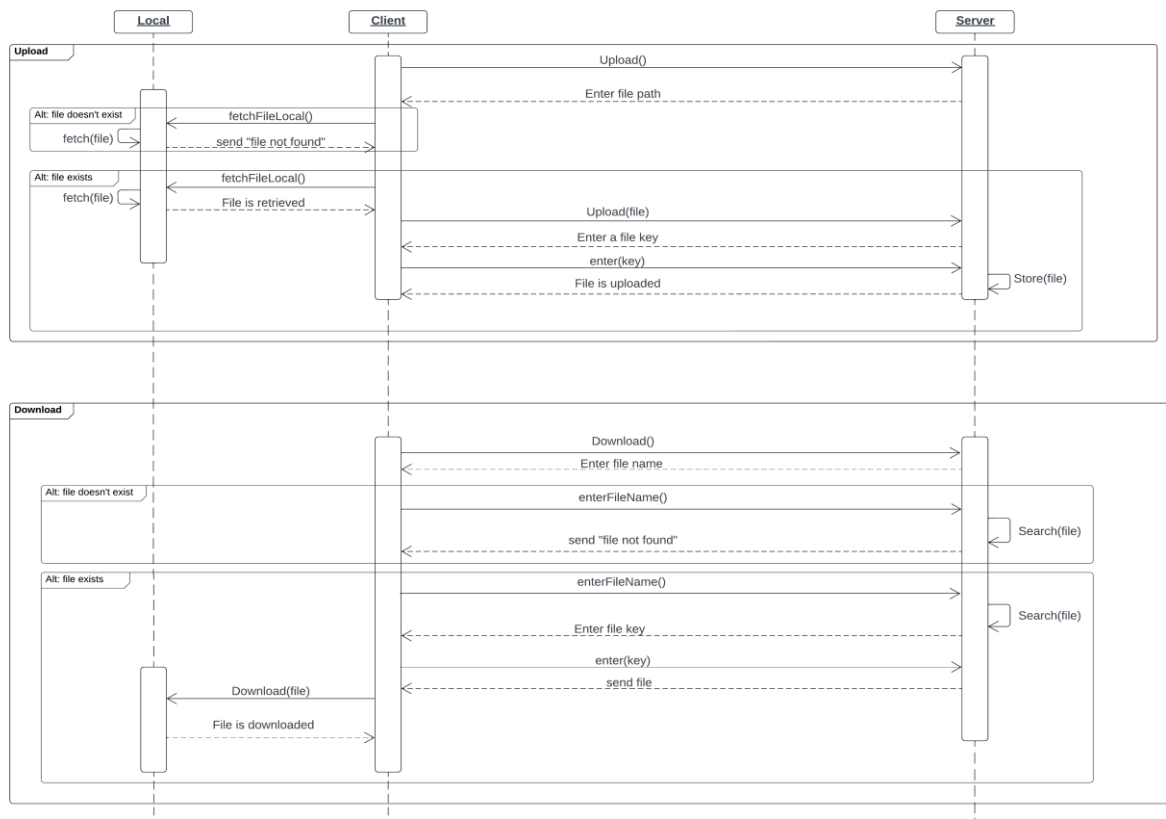
The protocol follows the standard structure. The messages include a payload and a header, with the header containing information such as the message type and message length. The payload contains the actual data being sent or received. The server listens on a specific port for incoming client connections and once the connection is established, the client and server exchange messages in a predefined sequence until the communication is complete. The server responds with appropriate responses when the client sends requests to the server. The specification of the protocol also deals with error handling procedures, such as defining lost or corrupted messages and network-related problems such as timeouts.

Sequence diagram

Client and Server without key (Public)



Client and Server with key (Private)



Screenshots of Features

Upload

Uploading a public file

```
Welcome to the file transfer protocol.
Commands:
exit - terminate connection
upload - upload a public or password protected file to the server
download - download a file from the server
list - print a list of server files

Enter a command:
> upload
Enter file path:
file1.txt
Set file key (or press enter to skip):

Successfully uploaded file to server.
```

Uploading a private file

```
Welcome to the file transfer protocol.
Commands:
exit - terminate connection
upload - upload a public or password protected file to the server
download - download a file from the server
list - print a list of server files

Enter a command:
> upload
Enter file path:
secrets.txt
Set file key (or press enter to skip):
password123
Successfully uploaded file to server.
```

Download

Downloading a public file

```
Welcome to the file transfer protocol.
Commands:
exit - terminate connection
upload - upload a public or password protected file to the server
download - download a file from the server
list - print a list of server files

Enter a command:
> download
Enter name of file:
file1.txt
Successfully downloaded file1.txt.
```

Downloading a private file

```
Welcome to the file transfer protocol.
Commands:
exit - terminate connection
upload - upload a public or password protected file to the server
download - download a file from the server
list - print a list of server files

Enter a command:
> download
Enter name of file:
secrets.txt
File is protected. Please provide the key:
qwerty
Incorrect file key provided.

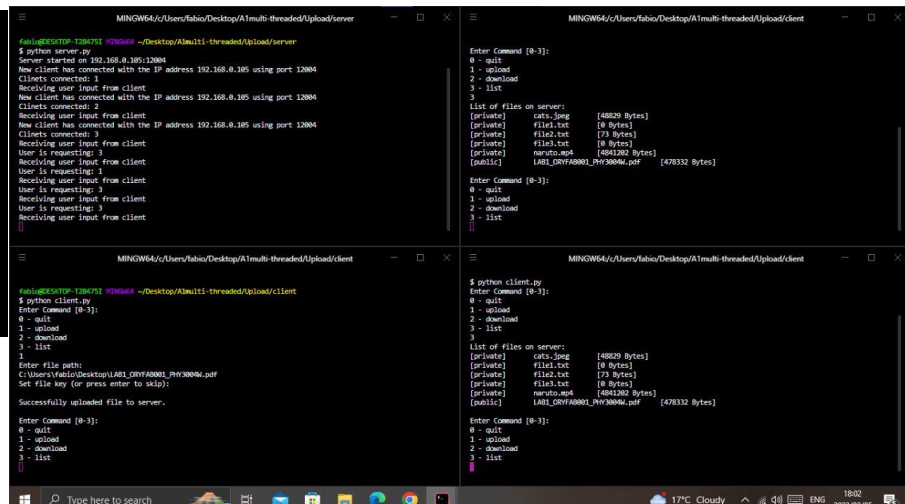
Enter a command:
> download
Enter name of file:
secrets.txt
File is protected. Please provide the key:
password123
Successfully downloaded secrets.txt.
```

List

```
Welcome to the file transfer protocol.
Commands:
exit - terminate connection
upload - upload a public or password protected file to the server
download - download a file from the server
list - print a list of server files

Enter a command:
> list
List of files on server:
[public]    file1.txt    [29 Bytes]
[private]   secrets.txt  [48 Bytes]
```

Multithreading



Stress Testing

Upload times on local device (measured using the time library in python):

1 byte - 0.000994 s

10 bytes - 0.038336 s

100 bytes - 0.000997 s

1KB - 0.000998 s

10KB - 0.010969 s

100 KB - 0.012180

1 MB - 0.014763 s

10MB - 0.101724 s

100MB - terminal crashed

1 GB - terminal crashed

For the bigger files there seems to be a linear relationship between file size and upload time. The larger the file the longer the uploading time. We can guarantee that the client can upload a file of 100MB or less with no problems. Files greater than 100MB may cause crashes. In a future version of our protocol the uploading implementation can be altered to send data to the server in chunks rather than all at once. This will allow for larger file uploads. Further stress testing can be done between a client and server on different networks to take network traffic into consideration.